

Lucrare de Laborator 4

Procesarea Semnalelor

Elaborat: Macrii Danu

Gr.: FAF-222

Chişinău, May 2025

Contents

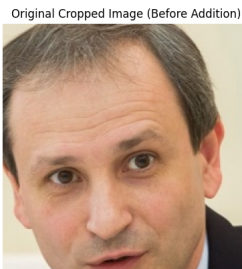
1	Manipularea Curbei Gama	3
1.1	Afișarea unei imagini la alegere	3
1.2	Adăugarea a 128 de valori la fiecare pixel	3
1.3	Scăderea a 128 de valori din fiecare pixel	3
1.4	Împărțirea la 2 a fiecărei valori de pixel	4
1.5	Înmulțirea cu 2 a fiecărei valori de pixel	4
1.6	Înmulțirea cu 0.5 și adăugarea a 128 de valori la fiecare pixel	4
1.7	Cercetarea transformării curbei gama (putere și logaritmică)	5
2	Egalarea Histogramei	5
2.1	Afișarea imaginii și a histogramei imaginii selectate	5
2.2	Egalarea histogramei și afișarea imaginii și histogramei egalizate	6
3	Filtrarea Imaginilor	6
3.1	Filtrarea imaginii cu un filtru 3x3	6
3.2	Filtrarea imaginii cu un filtru 5x7	7
3.3	Filtrarea imaginii cu un filtru 11x11	7
3.4	Detectarea marginilor imaginii cu un filtru Laplacian și Laplacian al Gaussianeii . . .	8
3.5	Adăugarea zgomotului "sare și piper" la imagine	8
3.6	Filtrarea imaginii cu un filtru 3x3 (zgomot)	9
4	Filtrarea în Domeniul de Frecvențe	10
4.1	Generarea unei imagini-figură geometrică și afișarea spectrului Fourier al acesteia . .	10
4.2	Adăugarea zgomotului "sare și piper" la imagine și afișarea spectrului Fourier	10
4.3	Filtrarea în frecvență – multiplicând imaginea din p. 4.1 cu spectrul din 4.2 – elim- inând frecvențele înalte	11
4.4	Transformarea Fourier inversă a spectrului imaginii trunchiate	12
	Conclusions	13

1 Manipularea Curbei Gama

1.1 Afișarea unei imagini la alegere

1. Condiția problemei: Afișați o imagine la alegerea D-stră utilizând imshow. În prealabil, salvați această imagine.
2. Fragment de cod:

```
cc = c[0:min(c.shape[0], 256), 0:min(c.shape[1], 256)] if c.shape[0] < 256 or c.shape[1]  
  
plt.figure()  
plt.imshow(cc)  
plt.title('Image original')  
plt.axis('off')
```

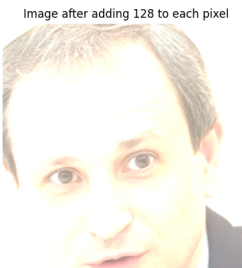


3. Imaginea rezultată:

1.2 Adăugarea a 128 de valori la fiecare pixel

1. Condiția problemei: Efectuați adăugarea la fiecare valoare de pixel din imagine a 128 de valori.
2. Fragment de cod:

```
cc = cc + 128
```



3. Imaginea rezultată:

1.3 Scăderea a 128 de valori din fiecare pixel

1. Condiția problemei: Efectuați scăderea la fiecare valoare de pixel din imagine a 128 de valori.
2. Fragment de cod:

```
cc = cc - 128
```

Image after adding 128 to each pixel



3. Imaginea rezultată:

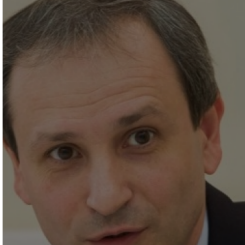
1.4 Împărțirea la 2 a fiecărei valori de pixel

1. Condiția problemei: Efectuați împărțirea la 2 a fiecărei valori de pixel din imagine.

2. Fragment de cod:

```
cc = cc / 2
```

Image after dividing by 2 pixel



3. Imaginea rezultată:

1.5 Înmulțirea cu 2 a fiecărei valori de pixel

1. Condiția problemei: Efectuați înmulțirea cu 2 a fiecărei valori de pixel din imagine.

2. Fragment de cod:

```
cc = cc * 2
```

Image after multiplying by 2 pixel



3. Imaginea rezultată:

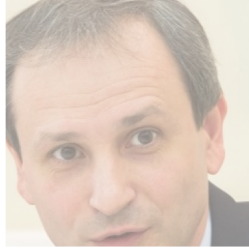
1.6 Înmulțirea cu 0.5 și adăugarea a 128 de valori la fiecare pixel

1. Condiția problemei: Efectuați înmulțirea cu 0.5 și adăugarea a 128 valori la fiecare valoare de pixel din imagine.

2. Fragment de cod: `cc = cc * 0.5 + 128`

```
cc = cc * 2
```

Image after multiplying by 0.5 and adding 128 pixel



Imaginea rezultată:

1.7 Cercetarea transformării curbei gama (putere și logaritmică)

1. Condiția problemei: Cercetați transformarea curbei gama, transformarea de putere și logaritmică accesând aplicația.
2. Fragment de cod:

```
cc_float = cc.astype(np.float32)

max_pixel_value = np.max(cc_float)
if max_pixel_value == 0:
    c_log = 0
else:
    c_log = 255 / np.log(1 + max_pixel_value)

c1_log_transformed = c_log * np.log(1 + cc_float)
```

Image after Logarithmic Scaling



3. Imaginea rezultată:

2 Egalarea Histogramei

2.1 Afișarea imaginii și a histogramei imaginii selectate

1. Condiția problemei: Afișați imaginea și histograma imaginii selectate.
2. Fragment de cod:

```
cc = c[50:306, 206:462]

cc_rgb = cv2.cvtColor(cc, cv2.COLOR_BGR2RGB) if len(cc.shape) == 3 else cc
```

```

colors = ('b', 'g', 'r')
for i, color in enumerate(colors):
    hist = cv2.calcHist([cc], [i], None, [256], [0, 256])
    plt.plot(hist, color=color, alpha=0.7)
plt.title('Histograms (Color Channels)')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')
plt.legend(['Blue channel', 'Green channel', 'Red channel'])
plt.grid(True)
plt.show()

```

3. Imaginea rezultată: 

2.2 Egalarea histogramei și afișarea imaginii și histogramei egalizate

1. Condiția problemei: Efectuați egalarea histogramei și afișați imaginea și histograma imaginii selectate.

2. Fragment de cod:

```
b, g, r = cv2.split(cc)
```

```

b_eq = cv2.equalizeHist(b)
g_eq = cv2.equalizeHist(g)
r_eq = cv2.equalizeHist(r)

```

```

h_color = cv2.merge([b_eq, g_eq, r_eq])
h_color_rgb = cv2.cvtColor(h_color, cv2.COLOR_BGR2RGB)

```

```

colors_eq = ('b', 'g', 'r')
for i, color in enumerate(colors_eq):
    hist_eq = cv2.calcHist([h_color], [i], None, [256], [0, 256])
    plt.plot(hist_eq, color=color, alpha=0.7)
plt.title('Histograms of Equalized Color')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')
plt.legend(['Blue channel (Equalized)', 'Green channel (Equalized)', 'Red channel (Equalized)'])
plt.grid(True)
plt.tight_layout()
plt.show()

```

3. Imaginea rezultată: 

3 Filtrarea Imaginilor

3.1 Filtrarea imaginii cu un filtru 3x3

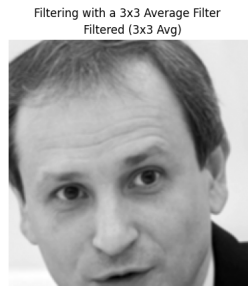
1. Condiția problemei: Efectuați filtrarea imaginii cu un filtru 3x3.

2. Fragment de cod:

```
kernel_3x3 = np.ones((3, 3), np.float32) / (3 * 3)

cf1_3x3_gray = cv2.filter2D(IG, -1, kernel_3x3)

plt.imshow(cf1_3x3_gray, cmap='gray')
plt.title('Filtered (3x3 Avg)')
plt.axis('off')
plt.suptitle('Filtering with a 3x3 Average Filter')
plt.show()
```



3. Imaginea rezultată:

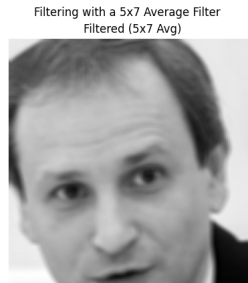
3.2 Filtrarea imaginii cu un filtru 5x7

1. Condiția problemei: Efectuați filtrarea imaginii cu un filtru 5x7.

2. Fragment de cod:

```
kernel_5x7 = np.ones((5, 7), np.float32) / (5 * 7)

cf1_5x7_gray = cv2.filter2D(IG, -1, kernel_5x7)
```



3. Imaginea rezultată:

3.3 Filtrarea imaginii cu un filtru 11x11

1. Condiția problemei: Efectuați filtrarea imaginii cu un filtru 11x11.

2. Fragment de cod:

```
kernel_11x11 = np.ones((11, 11), np.float32) / (11 * 11)

cf1_11x11_gray = cv2.filter2D(IG, -1, kernel_11x11)
```



3. Imaginea rezultată:

3.4 Detectarea marginilor imaginii cu un filtru Laplacian și Laplacian al Gaussianeii

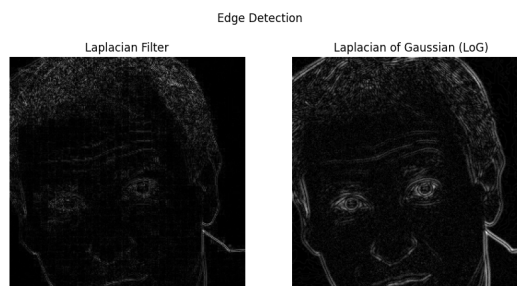
1. Condiția problemei: Efectuați detectarea mărginii imaginii cu un filtru Laplasian și Laplasian de la Gaussian.
2. Fragment de cod:

```
laplacian_kernel = np.array([[0, 1, 0],
                             [1, -4, 1],
                             [0, 1, 0]], dtype=np.float32)
cf2_laplacian = cv2.filter2D(IG, cv2.CV_32F, laplacian_kernel)
cf2_laplacian = np.uint8(np.clip(np.abs(cf2_laplacian), 0, 255))

sigma_log = 1.0
ksize_log = int(2 * np.ceil(2 * sigma_log) + 1)
if ksize_log % 2 == 0: ksize_log += 1

gaussian_blur_log = cv2.GaussianBlur(IG, (ksize_log, ksize_log), sigma_log)
cf3_log = cv2.Laplacian(gaussian_blur_log, cv2.CV_32F)
cf3_log = np.uint8(np.clip(np.abs(cf3_log), 0, 255))

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cf2_laplacian, cmap='gray')
plt.title('Laplacian Filter')
```



3. Imaginea rezultată:

3.5 Adăugarea zgomotului "sare și piper" la imagine

1. Condiția problemei: Efectuați adăugarea zgomotului "sare și piper" la imagine.

2. Fragment de cod:

```
def add_salt_and_pepper_noise(image, density=0.05):
    noisy_image = np.copy(image)
    total_pixels = image.size if len(image.shape) == 2 else image.shape[0] * image.shape[1]
    num_salt = int(total_pixels * density / 2)
    num_pepper = int(total_pixels * density / 2)

    coords = [np.random.randint(0, i - 1, num_salt) for i in image.shape]
    if len(image.shape) == 2: # Grayscale
        noisy_image[coords[0], coords[1]] = 255
    else: # Color (apply to all channels)
        noisy_image[coords[0], coords[1], :] = 255

    coords = [np.random.randint(0, i - 1, num_pepper) for i in image.shape]
    if len(image.shape) == 2: # Grayscale
        noisy_image[coords[0], coords[1]] = 0
    else: # Color (apply to all channels)
        noisy_image[coords[0], coords[1], :] = 0
```



3. Imaginea rezultată:

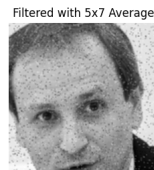
3.6 Filtrarea imaginii cu un filtru 3x3 (zgomot)

1. Condiția problemei: Filtrarea imaginii cu un filtru 3x3.

2. Fragment de cod:

```
kernel_a3 = np.ones((3, 3), np.float32) / (3 * 3)
c_sp_f3 = cv2.filter2D(c_sp_gray, -1, kernel_a3)
```

Filtering Salt & Pepper Noisy Image



3. Imaginea rezultată:

4 Filtrarea în Domeniul de Frecvențe

4.1 Generarea unei imagini-figură geometrică și afișarea spectrului Fourier al acesteia

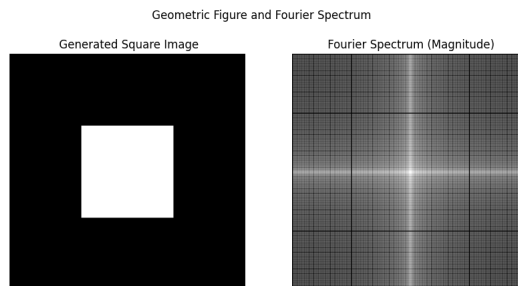
1. Condiția problemei: Generați o imagine-figură geometrică și afișați spectrul Fourier al acesteia.
2. Fragment de cod:

```
a = np.zeros((256, 256), dtype=np.uint8)
a[78:178, 78:178] = 255

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(a, cmap='gray')
plt.title('Generated Square Image')
plt.axis('off')

af_complex = np.fft.fftshift(np.fft.fft2(a))
af_magnitude = 20 * np.log(np.abs(af_complex) + 1)

plt.subplot(1, 2, 2)
plt.imshow(af_magnitude, cmap='gray')
plt.title('Fourier Spectrum (Magnitude)')
plt.axis('off')
plt.suptitle('Geometric Figure and Fourier Spectrum')
plt.show()
```



3. Imaginea rezultată:

4.2 Adăugarea zgomotului "sare și piper" la imagine și afișarea spectrului Fourier

1. Condiția problemei: Efectuați adăugarea zgomotului "sare și piper" la imagine și afișați spectrul Fourier.
2. Fragment de cod:

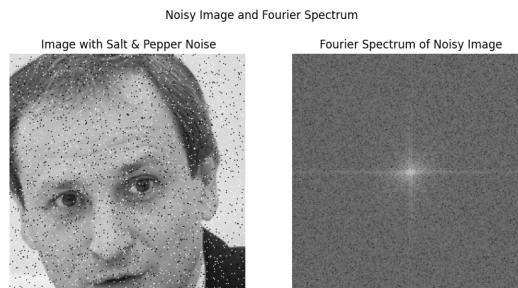
```
c_sp = add_salt_and_pepper_noise(IG, density=0.05)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
```

```
plt.imshow(c_sp, cmap='gray')
plt.title(' Image with Salt & Pepper Noise')
plt.axis('off')

cf_complex = np.fft.fftshift(np.fft.fft2(c_sp))
cf_magnitude = 20 * np.log(np.abs(cf_complex) + 1) # Log scale for visualization

plt.subplot(1, 2, 2)
plt.imshow(cf_magnitude, cmap='gray')
plt.title(' Fourier Spectrum of Noisy Image')
plt.axis('off')
plt.suptitle(' Noisy Image and Fourier Spectrum')
plt.show()
```



3. Imaginea rezultată:

4.3 Filtrarea în frecvență – multiplicând imaginea din p. 4.1 cu spectrul din 4.2 – eliminând frecvențele înalte

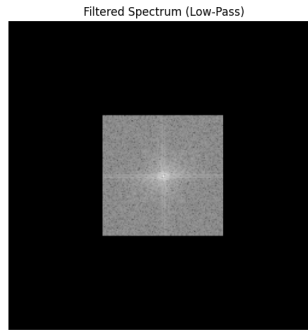
1. Condiția problemei: Efectuați filtrarea în frecvență – multiplicând imaginea din p. 4.1 cu spectrul din 4.2 – eliminând frecvențele înalte.
2. Fragment de cod:

```
low_pass_mask = a.astype(np.float32) / 255.0

cf1_filtered_complex = cf_complex * low_pass_mask

cf1_filtered_magnitude = 20 * np.log(np.abs(cf1_filtered_complex) + 1)

plt.figure(figsize=(6, 6))
plt.imshow(cf1_filtered_magnitude, cmap='gray')
plt.title(' Filtered Spectrum (Low-Pass)')
plt.axis('off')
```



3. Imaginea rezultată:

4.4 Transformarea Fourier inversă a spectrului imaginii trunchiate

1. Condiția problemei: Efectuați transformarea Fourier inversă a spectrului imaginii trunchiat.
2. Fragment de cod:

```
cf2_filtered_image = np.fft.ifft2(np.fft.ifftshift(cf1_filtered_complex)).real

cf2_display = np.clip(cf2_filtered_image, 0, 255).astype(np.uint8)

plt.figure(figsize=(6, 6))
plt.imshow(cf2_display, cmap='gray')
plt.title('Reconstructed Image after Low-Pass Filtering')
plt.axis('off')
plt.show()
```



3. Imaginea rezultată:

Conclusions

Acest raport a prezentat procedurile și rezultatele obținute în cadrul lucrărilor de laborator privind procesarea digitală a imaginilor. Diverse tehnici de manipulare a imaginilor au fost implementate și analizate. Acestea au inclus ajustarea curbei gama, egalizarea histogramelor, filtrarea spațială și filtrarea în domeniul frecvență. Fiecare secțiune a raportului a documentat condițiile problemei, implementarea algoritmilor prin secvențe de cod și efectul vizual al transformărilor aplicate. Rezultatele obținute demonstrează impactul specific al fiecărei tehnici asupra caracteristicilor imaginii, evidențiind modificări în contrast, distribuția intensității pixelilor, nivelul de zgomot și claritatea marginilor.